MINDSET IT SOLUTIONS AND CONSULTANTS INTERNSHIP
Harshith V -1MJ22CS073
MINIPROJECT ON FILE OPERATIONS

Project Title:

Java File Operations Console App with Login & Logging

1. Overview

This project is a console-based file management application written in Java.
Before accessing file operations, users must pass a simple authentication step. All login attempts (successful and failed) are logged to disk, and the app provides a summary of login statistics.

Key capabilities:

- Authentication gate before any file operations.
- File operations:
  - Create a file (with multi-line content)
  - Read a file
  - Append to a file
  - Delete a file
- Folder operations:
  - Create a folder
  - Remove an (empty) folder
  - List contents of a folder
- Login logging & statistics:
  - Log successful and failed login attempts (with timestamp and username)
  - Display all logs and aggregated counts

---

2. Project Structure

- `FileOperationsApp.java`
  Entry point (`main` method) and main menu/flow controller.

- `AuthManager.java`
  Handles user login and integrates with `LogManager` to log attempts.

- `LogManager.java`
  Responsible for creating log directory and files, and for recording and listing login attempts & statistics.

- `FileManager.java`
  Contains all file and folder operations used by the menu.

- `logs/` folder (runtime)
  - `login_success.log`
  - `login_failure.log`
  Created automatically if not present, used to store login attempt history.

---

3. Technologies Used

- Language: Java (standard SE, no external libraries)
- I/O APIs: `java.nio.file` (`Files`, `Path`, `Paths`, `StandardOpenOption`)
- Console input: `java.util.Scanner`
- Date/time: `java.time.LocalDateTime`, `java.time.format.DateTimeFormatter`
- Character encoding: `StandardCharsets.UTF_8`

No external dependencies or build tools are required besides a JDK.

---

4. Application Flow

1. Startup
   - `FileOperationsApp.main` is invoked.
   - Instances of:
     - `LogManager`
     - `AuthManager` (with a `LogManager` reference)
     - `FileManager`
     are created.
   - A `Scanner` is opened on `System.in`.

2. Authentication Phase
   - `AuthManager.performLogin(scanner)` is called.
   - The user has up to 5 attempts to enter the correct username and password.
   - Valid credentials:
     - Username: `USER1`
     - Password: `USER1`
   - For each attempt:
     - On success:
       - `LogManager.logSuccess(username)` is called.
       - `"Login successful."` is printed.
       - The app proceeds to the menu.
     - On failure:
       - Attempt counter is incremented.
       - `LogManager.logFailure(username)` is called.
       - The user sees `"Invalid credentials. Attempt X of 5."`.
   - If all 5 attempts fail:
     - The app prints `"Too many failed attempts. Access restricted."`

- The app exits.

3. Main Menu (File Operations)
   - After a successful login, `showMenu(scanner, logManager, fileManager)` is invoked.
   - The menu runs in an infinite loop until the user chooses to quit.
   - Options:
     1. Create a File
     2. Read a File
     3. Append to a File
     4. Create a Folder
     5. Remove Folder
     6. Delete a File
     7. Show Contents of a Folder
     8. Show Login Logs & Statistics
     9. Quit

4. Input Validation
   - Menu choice is read as a `String`, then parsed to `int`.
   - If parsing fails, the user is asked to enter a valid number between 1 and 9.
   - A `switch` statement calls the corresponding method on `FileManager` or `LogManager`, or exits on choice 9.

---

5. Detailed Class Documentation

5.1 `FileOperationsApp`

Responsibilities:

- Start the application.
- Coordinate authentication.
- Drive the main menu loop.

Key methods:

- `public static void main(String[] args)`
  - Initializes:
    - `LogManager logManager`
    - `AuthManager authManager`
    - `FileManager fileManager`
  - Opens `Scanner` using try-with-resources.
  - Calls `authManager.performLogin(scanner)`.
  - If login fails after 5 attempts, prints an access restriction message and exits.
  - If login succeeds, calls `showMenu(...)`.

- `private static void showMenu(Scanner scanner, LogManager logManager, FileManager fileManager)`

- Repeatedly:
  - Displays the menu.
  - Reads and validates an integer choice.
  - Uses a `switch` to call:
    - `fileManager.createFile(scanner)`
    - `fileManager.readFile(scanner)`
    - `fileManager.appendToFile(scanner)`
    - `fileManager.createFolder(scanner)`
    - `fileManager.removeFolder(scanner)`
    - `fileManager.deleteFile(scanner)`
    - `fileManager.showFolderContents(scanner)`
    - `logManager.showLoginLogsAndStats()`
  - Exits loop and program when the user chooses 9.

5.2 `AuthManager`

Purpose:
Encapsulates the login logic, including credential checking and communication with `LogManager`.

Fields

- `private static final String VALID_USERNAME = "USER1";`
- `private static final String VALID_PASSWORD = "USER1";`
- `private final LogManager logManager;` – used to log success/failure.

Constructor

- `public AuthManager(LogManager logManager)`
  - Stores a reference to the provided `LogManager`.

Methods:

- public boolean performLogin(Scanner scanner
  - Allows up to 5 login attempts in a while-loop.
  - For each attempt:
    - Prompts for username and password.
    - Checks against `VALID_USERNAME` and `VALID_PASSWORD`.
    - On success:
      - Calls `logManager.logSuccess(username)`.
      - Prints `"Login successful."`.
      - Returns `true`.
    - On failure:
      - Increments attempt count.
      - Calls `logManager.logFailure(username)`.
      - Prints an error message with current attempt number.

  - After 5 failed attempts, returns `false`.

5.3 `LogManager`

Purpose:
Manages **logging of login attempts** and presents **statistics and log contents**.

Fields:

- `private final Path logDir;` – path to `logs` directory.
- `private final Path successLog;` – `logs/login_success.log`.
- `private final Path failureLog;` – `logs/login_failure.log`.
- `private final DateTimeFormatter formatter;` – `"yyyy-MM-dd HH:mm:ss"`.

Constructor:

- `public LogManager()`
  - Initializes `logDir` (`"logs"`).
  - Sets up `successLog` and `failureLog` paths.
  - Initializes a timestamp formatter.
  - Calls `ensureLogDirectory()`.

Methods:

- private void ensureLogDirectory()
  - Creates the `logs` directory if it does not exist, using `Files.createDirectories`.
  - If directory creation fails, prints a warning.

- public void logSuccess(String username)
  - Delegates to `logLoginAttempt(successLog, username, true)`.

- public void logFailure(String username)
  - Delegates to `logLoginAttempt(failureLog, username, false)`.

- private void logLoginAttempt(Path logFile, String username, boolean success)
  - Builds a log line in the format:
    `yyyy-MM-dd HH:mm:ss - SUCCESS/FAILURE - user=<username>`
  - Ensures the log file exists (creates if necessary).
  - Appends the line using `Files.write` with `StandardOpenOption.APPEND`.
  - If an error occurs, prints a warning but does not crash the app.

- public void showLoginLogsAndStats()
  - Prints headings for Login Logs & Statistics.
  - Initializes counters `successCount` and `failureCount`.
  - For success log:
    - If file exists:

    - Reads all lines.
    - `successCount = number of lines`.
    - Prints each line or a "no attempts" message.
  - If file does not exist:
    - Prints `"No success log file found yet."`
- For **failure log:
  - Same pattern as success log; uses `failureCount`.
- Prints a **summary**:
  - Total successful login attempts
  - Total failed login attempts

5.4 `FileManager`

Purpose:
Implements all **file and directory operations** used by the menu.

Common patterns:

- Uses `Scanner` to get user input.
- Uses `Paths.get(...)` to turn user input into `Path`.
- Validates:
  - Non-empty names.
  - Existence or non-existence where appropriate.
- Uses `Files` methods (`readAllLines`, `write`, `delete`, `list`, etc.).
- All I/O operations are wrapped in `try-catch` blocks with user-friendly messages.

Methods:

- **`public void createFile(Scanner scanner)`**
  - Asks for a file name (relative or absolute).
  - Validates non-empty; returns if empty.
  - Builds a `Path`.
  - Prompts user to enter content; the user types multiple lines.
  - Terminates input when user enters a line `"EOF"` (case-insensitive).
  - Ensures parent directories exist (creates them if needed).
  - Writes the collected content to file in UTF-8.
  - Prints the absolute path on success.

- **`public void readFile(Scanner scanner)`**
  - Asks for a file name.
  - Validates non-empty; returns if empty.
  - Validates that the file exists.
  - Reads all lines using `Files.readAllLines`.
  - Prints `"----- File Contents -----"`, then each line, then `"----- End of File -----"`.

- **`public void appendToFile(Scanner scanner)`**

  - Asks for a file name.
  - Validates non-empty and existence.
  - Prompts for content to append, ending on `"EOF"` line.
  - Appends the collected content using `StandardOpenOption.APPEND`.
  - Prints success or error message.

- **`public void createFolder(Scanner scanner)`**
  - Asks for folder name (relative or absolute).
  - Validates non-empty.
  - Creates all necessary directories via `Files.createDirectories` if folder does not yet exist.
  - If folder already exists, prints an info message and does nothing.
  - On success, prints the absolute path.

- **`public void removeFolder(Scanner scanner)`**
  - Asks for folder name.
  - Validates non-empty.
  - Validates that path exists.
  - Attempts `Files.delete(folderPath)`.
  - If folder is not empty, catches `DirectoryNotEmptyException` and shows an appropriate message.
  - Handles other `IOException`s with an error message.

- **`public void deleteFile(Scanner scanner)`**
  - Asks for file name.
  - Validates non-empty.
  - Ensures the file exists.
  - Deletes the file via `Files.delete`.
  - Displays success or error message.

- **`public void showFolderContents(Scanner scanner)`**
  - Prompts for folder path.
  - If the user leaves it empty, uses the **current working directory** (`"."`) and normalizes it.
  - If path is provided, uses that path.
  - Validates that path exists and is a directory.
  - Uses `Files.list` to print each entry as:
    - `"[DIR] <name>"` for directories
    - `"[FILE] <name>"` for regular files

---

6. How to Run the Project

1. **Requirements**
   - JDK installed (e.g., Java 8+).
   - Command line access (e.g., PowerShell on Windows, as you have).

2. **Compile**

From the project root (e.g., `Java_miniproject1`), run:

```bash
javac -d out src/*.java
```

This compiles all `.java` files into the `out` directory.

3. **Run**

```bash
java -cp out FileOperationsApp
```

4. Login

- When prompted:
  - Username: `USER1`
  - Password: `USER1`
- After successful login, the menu is displayed.

---

7. Login Log Files Format

At runtime, `LogManager` creates a `logs` folder (if missing) with:

- `login_success.log`
- `login_failure.log`

Each line looks like:

```text
2026-02-25 14:12:34 - SUCCESS - user=USER1
```

or

```text
2026-02-25 14:13:01 - FAILURE - user=wrongUser
```

You can view these via **Menu option 8** or by opening the log files directly.

---

8. Error Handling & Edge Cases

- **Invalid menu input** (non-number):
  - User is asked to enter a valid number between 1 and 9.
- **Empty names** for files/folders:
  - Operation is canceled with a message.
- **Non-existent paths**:
  - For reading, appending, deleting files, and removing/listing folders, the code checks for existence and prints helpful errors instead of throwing.
- **Non-empty folder removal**:
  - Specifically handled with `DirectoryNotEmptyException` and a clear message.
- **Logging failures**:
  - If log files or folders cannot be created/written, a warning is printed; the app continues running.

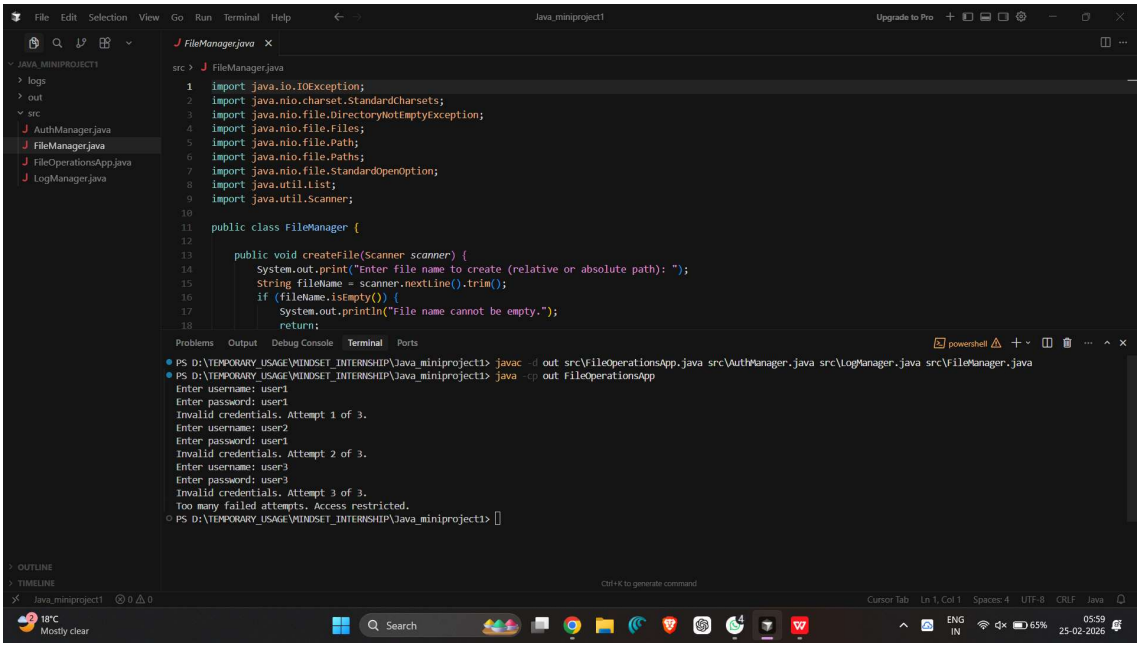
---

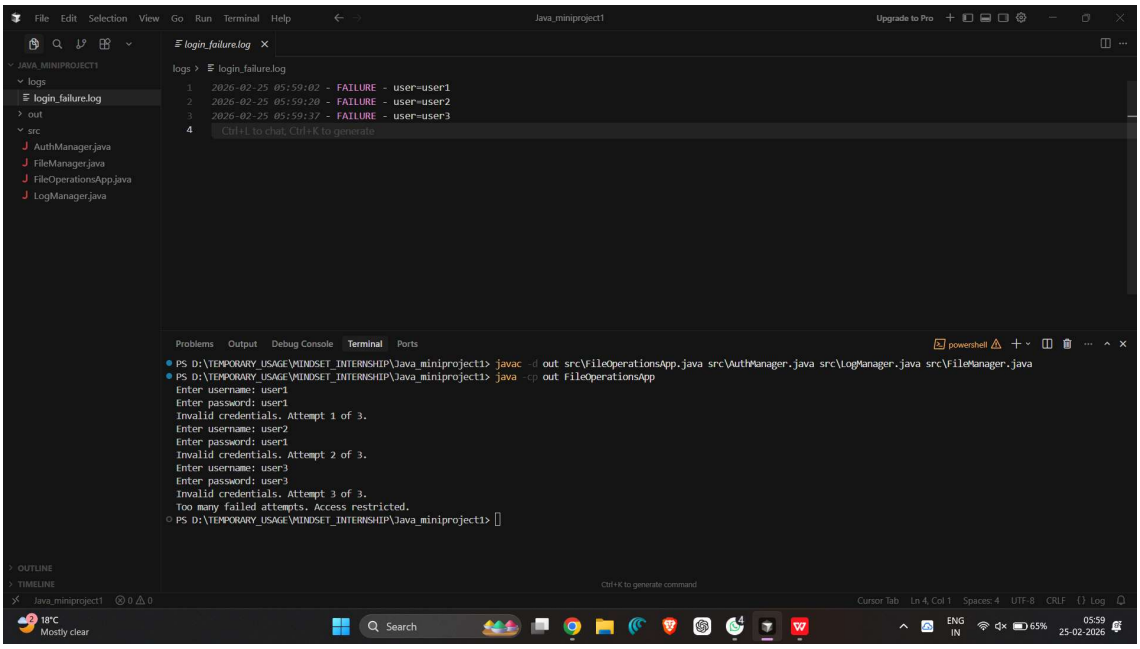
9. Possible Extensions

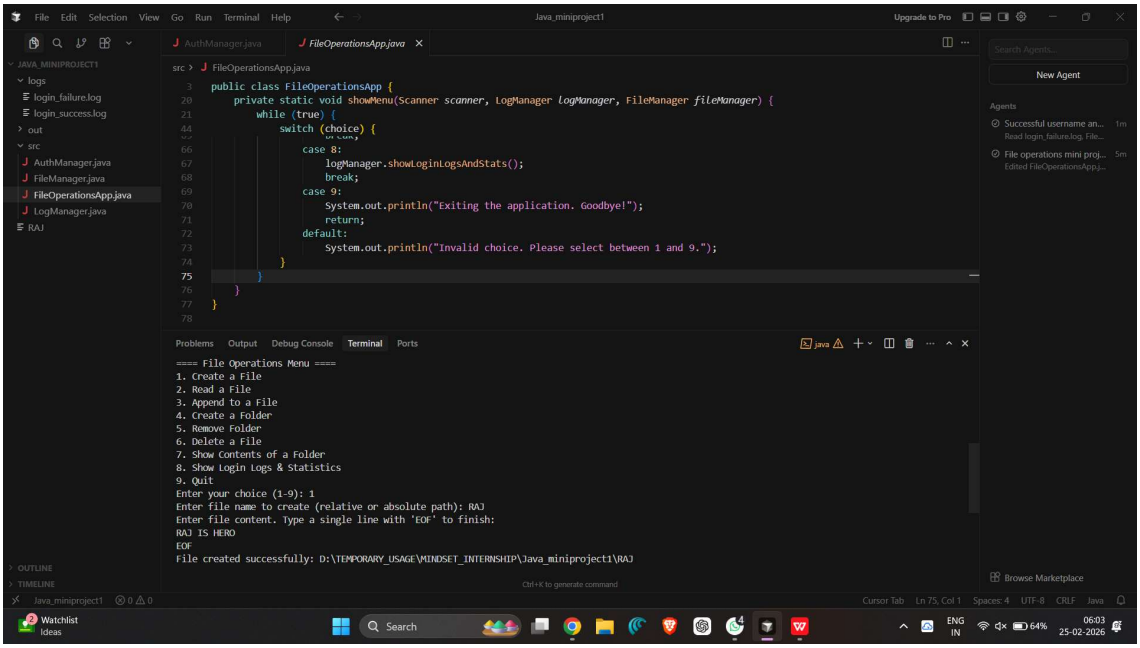
If you want to expand this project, some natural next steps are:

- **Configurable credentials** (read from a file instead of hardcoded).
- **Password masking** when entering passwords in console.
- **Role-based access** (read-only vs full access).
- **More file operations** (copy, move, rename).
- **Search in files** or filter folder contents.
- **Unit tests** for `FileManager`, `AuthManager`, and `LogManager`.

10.Results:




File Edit Selection View Go Run Terminal Help
Java_miniproject1
Upgrade to Pro
JAVA_MINIPROJECT1
logs
login_failure.log
out
src
AuthManager.java
FileManager.java
FileOperationsApp.java
LogManager.java
OUTLINE
TIMELINE
logs > login_failure.log
2026-02-25 05:59:02 - FAILURE - user=user1
2026-02-25 05:59:20 - FAILURE - user=user2
2026-02-25 05:59:37 - FAILURE - user=user3
Ctrl+L to chat, Ctrl+K to generate
Problems Output Debug Console Terminal Ports
powershell
PS D:\TEMPORARY_USAGE\MINDSET_INTERNSHIP\Java_miniproject1> javac -d out src\FileOperationsApp.java src\AuthManager.java src\LogManager.java src\FileManager.java
PS D:\TEMPORARY_USAGE\MINDSET_INTERNSHIP\Java_miniproject1> java -cp out FileOperationsApp
Enter username: user1
Enter password: user1
Invalid credentials. Attempt 1 of 3.
Enter username: user2
Enter password: user1
Invalid credentials. Attempt 2 of 3.
Enter username: user3
Enter password: user3
Invalid credentials. Attempt 3 of 3.
Too many failed attempts. Access restricted.
PS D:\TEMPORARY_USAGE\MINDSET_INTERNSHIP\Java_miniproject1>
Ctrl+K to generate command
Cursor Tab Ln 4, Col 1 Spaces: 4 UTF-8 CRLF {} Log
18°C
Mostly clear
Search
ENG
IN
65%
05:59
25-02-2026


File Edit Selection View Go Run Terminal Help
Java_miniproject1
Upgrade to Pro
AuthManager.java
FileOperationsApp.java
JAVA_MINIPROJECT1
logs
login_failure.log
login_success.log
out
src
AuthManager.java
FileManager.java
FileOperationsApp.java
LogManager.java
RAJ
OUTLINE
TIMELINE
src > FileOperationsApp.java
public class FileOperationsApp {
private static void showMenu(Scanner scanner, LogManager logManager, FileManager fileManager) {
while (true) {
switch (choice) {
case 8:
logManager.showLoginLogsAndStats();
break;
case 9:
System.out.println("Exiting the application. Goodbye!");
return;
default:
System.out.println("Invalid choice. Please select between 1 and 9.");
}
}
}
}
Problems Output Debug Console Terminal Ports
java
==== File Operations Menu ====
1. Create a File
2. Read a File
3. Append to a File
4. Create a Folder
5. Remove Folder
6. Delete a File
7. Show Contents of a Folder
8. Show Login Logs & Statistics
9. Quit
Enter your choice (1-9): 1
Enter file name to create (relative or absolute path): RAJ
Enter file content. Type a single line with 'EOF' to finish:
RAJ IS HERO
EOF
File created successfully: D:\TEMPORARY_USAGE\MINDSET_INTERNSHIP\Java_miniproject1\RAJ
Ctrl+K to generate command
New Agent
Agents
Successful username an...
Read login_failure.log, File...
File operations mini proj...
Edited FileOperationsApp.j...
Browse Marketplace
Cursor Tab Ln 75, Col 1 Spaces: 4 UTF-8 CRLF Java
Watchlist
Ideas
Search
ENG
IN
64%
06:03
25-02-2026

File Edit Selection View Go Run Terminal Help — Java_miniproject1 — Upgrade to Pro

AuthManager.java | login_success.log

logs > login_success.log

```
2026-02-25 06:03:01 - SUCCESS - user=USER1
```

Problems Output Debug Console Terminal Ports

```
==== File Operations Menu ====
1. Create a File
2. Read a File
3. Append to a File
4. Create a Folder
5. Remove Folder
6. Delete a File
7. Show Contents of a Folder
8. Show Login Logs & Statistics
9. Quit
Enter your choice (1-9): 1
Enter file name to create (relative or absolute path): RAJ
Enter file content. Type a single line with 'EOF' to finish:
RAJ IS HERO
EOF
File created successfully: D:\TEMPORARY_USAGE\MINDSET_INTERNSHIP\Java_miniproject1\RAJ
```

JAVA_MINIPRO...
- logs
  - login_failure.log
  - login_success.log
- out
- src
  - AuthManager.java
  - FileManager.java
  - FileOperationsApp.java
  - LogManager.java
- RAJ

OUTLINE
TIMELINE

Search Agents... | New Agent

Agents
- Successful username an... 1m — Read login_failure.log. File...
- File operations mini proj... 5m — Edited FileOperationsApp.j...

Browse Marketplace

Ctrl+K to generate command

Java_miniproject1 ⊗ 0 ⚠ 0 — Cursor Tab — Ln 1, Col 1 — Spaces: 4 — UTF-8 — CRLF — Log

Watchlist Ideas — Search — ENG IN — 63% — 06:03 25-02-2026

File Edit Selection View Go Run Terminal Help — Java_miniproject1 — Upgrade to Pro

AuthManager.java | login_success.log

logs > login_success.log

Problems Output Debug Console Terminal Ports

```
==== File Operations Menu ====
1. Create a File
2. Read a File
3. Append to a File
4. Create a Folder
5. Remove Folder
6. Delete a File
7. Show Contents of a Folder
8. Show Login Logs & Statistics
9. Quit
Enter your choice (1-9): 2
Enter file name to read: RAJ
----- File Contents -----
RAJ IS HERO
----- End of File -----
```

JAVA_MINIPROJECT1
- logs
  - login_failure.log
  - login_success.log
- out
- src
  - AuthManager.java
  - FileManager.java
  - FileOperationsApp.java
  - LogManager.java
- RAJ

OUTLINE
TIMELINE

Search Agents... | New Agent

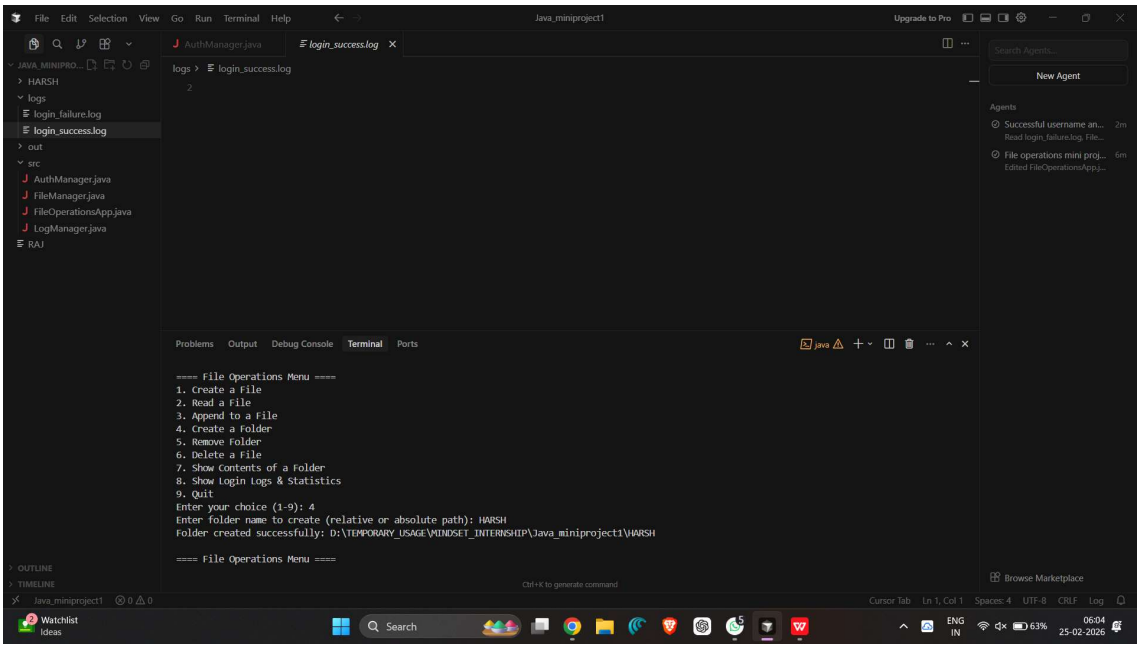
Agents
- Successful username an... 1m — Read login_failure.log. File...
- File operations mini proj... 5m — Edited FileOperationsApp.j...

Browse Marketplace

Ctrl+K to generate command

Java_miniproject1 ⊗ 0 ⚠ 0 — Cursor Tab — Ln 1, Col 1 — Spaces: 4 — UTF-8 — CRLF — Log

Watchlist Ideas — Search — ENG IN — 63% — 06:04 25-02-2026


File Edit Selection View Go Run Terminal Help
Java_miniproject1
Upgrade to Pro
AuthManager.java
login_success.log
logs > login_success.log
JAVA_MINIPRO...
HARSH
logs
login_failure.log
login_success.log
out
src
AuthManager.java
FileManager.java
FileOperationsApp.java
LogManager.java
RAJ
Problems Output Debug Console Terminal Ports
java
==== File Operations Menu ====
1. Create a File
2. Read a File
3. Append to a File
4. Create a Folder
5. Remove Folder
6. Delete a File
7. Show Contents of a Folder
8. Show Login Logs & Statistics
9. Quit
Enter your choice (1-9): 4
Enter folder name to create (relative or absolute path): HARSH
Folder created successfully: D:\TEMPORARY_USAGE\MINDSET_INTERNSHIP\Java_miniproject1\HARSH
==== File Operations Menu ====
OUTLINE
TIMELINE
Ctrl+K to generate command
Search Agents...
New Agent
Agents
Successful username an... 2m
Read login_failure.log, File...
File operations mini proj... 6m
Edited FileOperationsApp.j...
Browse Marketplace
Java_miniproject1
Cursor Tab Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Log
Watchlist
Ideas
Search
ENG
IN
63%
06:04
25-02-2026